

Active Recognition Roster

LIONEL MESSI
Football

VIRAT KOHLI
Cricket

ROGER FEDERER
Tennis

SERENA WILLIAMS
Tennis

MARIA SHARAPOVA
Tennis

```
import numpy as np
import cv2
import matplotlib.pyplot as plt
```

cv2 will read image

```
from google.colab import drive
drive.mount('/content/drive')
```

Mounted at /content/drive

```
img = cv2.imread('/content/drive/My Drive/Colab Notebooks/Sportsperson image classification supervised learning/Sports person c:
img.shape

(1038, 1600, 3)
```

```
plt.imshow(img)
```

<matplotlib.image.AxesImage at 0x7b5723a8b950>



```
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
gray.shape
```

(1038, 1600)

```
gray
```

```
array([[44, 44, 44, ..., 52, 52, 52],
       [44, 44, 44, ..., 52, 52, 52],
       [44, 44, 44, ..., 52, 52, 52],
       ...,
       [19, 19, 19, ..., 23, 23, 23],
       [19, 19, 19, ..., 23, 23, 23],
       [19, 19, 19, ..., 23, 23, 23]], dtype=uint8)
```

```
plt.imshow(gray, cmap='gray')
```

```
<matplotlib.image.AxesImage at 0x7b571077a150>
```



```
face_cascade = cv2.CascadeClassifier('/content/drive/My Drive/Colab Notebooks/Sportsperson image classification supervised learn
eye_cascade = cv2.CascadeClassifier('/content/drive/My Drive/Colab Notebooks/Sportsperson image classification supervised learn

faces =face_cascade.detectMultiScale(gray, 1.3, 5)
faces
```

```
array([[670, 285, 671, 671]], dtype=int32)
```

x,y , width, height

```
x,y,w,h = faces[0]
x,y,w,h
```

```
(np.int32(670), np.int32(285), np.int32(671), np.int32(671))
```

```
face_img = cv2.rectangle(img,(x,y), (x+w, y+h),(255,0,0),2)
plt.imshow(face_img)
```

```
<matplotlib.image.AxesImage at 0x7b57107db0e0>
```



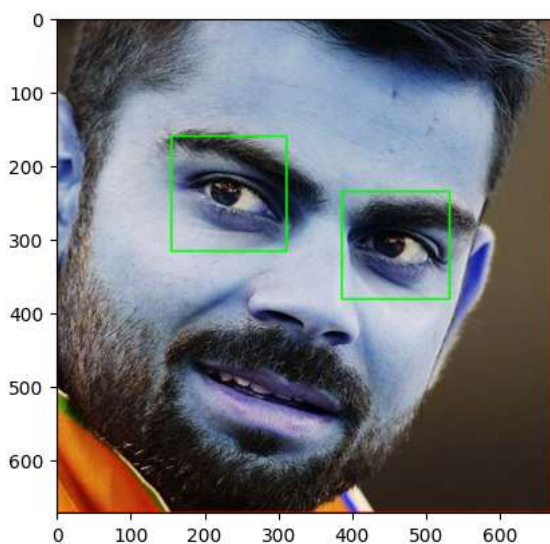
```
cv2.destroyAllWindows()
for(x,y,w,h) in faces:
    face_img = cv2.rectangle(img,(x,y),(x+w,y+h), (255,0,0),2)
    roi_gray = gray[y:y+h, x:x+w]
    roi_color = face_img [y:y+h, x:x+w]
    eyes = eye_cascade.detectMultiScale(roi_gray)
    for(ex,ey,ew,eh) in eyes:
        cv2.rectangle(roi_color,(ex,ey),(ex+ew,ey+eh),(0,255,0),2)

plt.figure( )
plt.imshow(face_img, cmap='gray')
plt.show()
```



```
plt.imshow(roi_color, cmap='gray')
```

```
<matplotlib.image.AxesImage at 0x7b571035bef0>
```



will use this function for my all images

```
def get_cropped_image_if_2_eyes(image_path):
    img = cv2.imread(image_path)
    gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
    faces = face_cascade.detectMultiScale(gray, 1.3, 5)
    for(x,y,w,h) in faces :
        roi_gray= gray[y:y+h, x:x+w]
        roi_color = img[y:y+h, x:x+w]
        eyes = eye_cascade.detectMultiScale(roi_gray)
        if len(eyes) >=2 :
            return roi_color
```

```
original_image = cv2.imread('/content/drive/My Drive/Colab Notebooks/Sportsperson image classification supervised learningng/Spor')
plt.imshow(original_image)
```

```
<matplotlib.image.AxesImage at 0x7b570ff8b7a0>
```



```
image_path = '/content/drive/My Drive/Colab Notebooks/Sportsperson image classification supervised learning/Sports person clas
cropped_image_data = get_cropped_image_if_2_eyes(image_path)
```

```
if cropped_image_data is not None:
    plt.imshow(cropped_image_data)
    plt.title("Cropped Image")
    plt.axis('off')
    plt.show()
else:
    print(f"Could not find face with 2 eyes in the image: {image_path}")
```

Cropped Image



```
path_to_data = '/content/drive/My Drive/Colab Notebooks/Sportsperson image classification supervised learning/Sports person cl
path_to_cr_data = '/content/drive/My Drive/Colab Notebooks/Sportsperson image classification supervised learning/Sports person
```

```
import os
img_dirs = []
for entry in os.scandir(path_to_data):
    # Exclude 'cropped' and 'cropped1' directories
    if entry.is_dir() and entry.name not in ('cropped', 'cropped1'):
        img_dirs.append(entry.path)
```

```
img_dirs
```

```
['/content/drive/My Drive/Colab Notebooks/Sportsperson image classification supervised learning/Sports person
classifier/model/dataset/virat_kohli',
'/content/drive/My Drive/Colab Notebooks/Sportsperson image classification supervised learning/Sports person
classifier/model/dataset/serena_williams',
'/content/drive/My Drive/Colab Notebooks/Sportsperson image classification supervised learning/Sports person
classifier/model/dataset/roger_federer',
'/content/drive/My Drive/Colab Notebooks/Sportsperson image classification supervised learning/Sports person
classifier/model/dataset/maria_sharapova',
```

[illegible]

Feature Engineering

```
import numpy as np
import pywt
import cv2

def w2d(img, mode='haar', level=1):
    imArray = img
    #Datatype conversion
    #Convert to grayscale
    imArray = cv2.cvtColor(imArray, cv2.COLOR_RGB2GRAY)
    #Convert to float
    imArray = np.float32(imArray)
    imArray /= 255
    #Compute coefficients
    coeffs = pywt.wavedec2(imArray, mode, level=level)

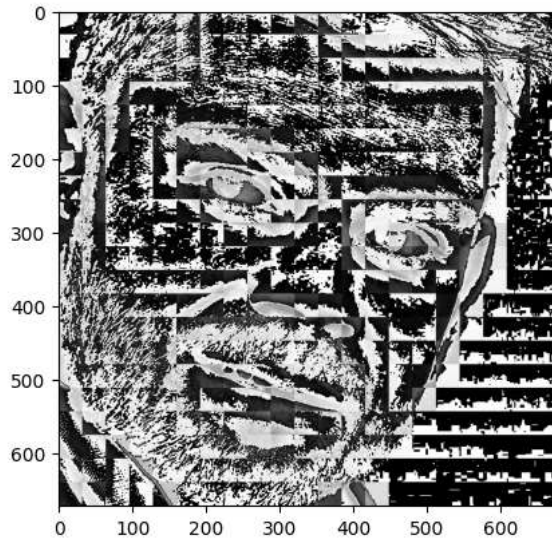
    #Process coefficients
    coeffs_H = list(coeffs)
    coeffs_H[0] *= 0

    #Reconstruct image
    imArray_H = pywt.waverec2(coeffs_H, mode)
    imArray_H *= 255
    imArray_H = np.uint8(imArray_H)

    return imArray_H
```

6/11

<matplotlib.image.AxesImage at 0x7b570f940aa0>



celebrity_file_names_dict

```
{'virat_kohli': ['/content/drive/My Drive/Colab Notebooks/Sportsperson image classification supervised learningnng/Sports person
classifier/model/dataset/virat_kohli/6adbc1218c.jpg',
'/content/drive/My Drive/Colab Notebooks/Sportsperson image classification supervised learningnng/Sports person
classifier/model/dataset/virat_kohli/5a3384f779.jpg',
'/content/drive/My Drive/Colab Notebooks/Sportsperson image classification supervised learningnng/Sports person
classifier/model/dataset/virat_kohli/1fee075645.jpg',
'/content/drive/My Drive/Colab Notebooks/Sportsperson image classification supervised learningnng/Sports person
classifier/model/dataset/virat_kohli/3e6d62bc8e.jpg',
'/content/drive/My Drive/Colab Notebooks/Sportsperson image classification supervised learningnng/Sports person
classifier/model/dataset/virat_kohli/4cbef36ff6.jpg',
'/content/drive/My Drive/Colab Notebooks/Sportsperson image classification supervised learningnng/Sports person
classifier/model/dataset/virat_kohli/02fa06cf94.jpg',
'/content/drive/My Drive/Colab Notebooks/Sportsperson image classification supervised learningnng/Sports person
classifier/model/dataset/virat_kohli/3f5fd4fead.jpg',
'/content/drive/My Drive/Colab Notebooks/Sportsperson image classification supervised learningnng/Sports person
classifier/model/dataset/virat_kohli/05b9bac90a.jpg',
'/content/drive/My Drive/Colab Notebooks/Sportsperson image classification supervised learningnng/Sports person
classifier/model/dataset/virat_kohli/0b43460429.jpg',
'/content/drive/My Drive/Colab Notebooks/Sportsperson image classification supervised learningnng/Sports person
classifier/model/dataset/virat_kohli/0eb2230fbf.jpg',
'/content/drive/My Drive/Colab Notebooks/Sportsperson image classification supervised learningnng/Sports person
classifier/model/dataset/virat_kohli/0b53cce8c6.jpg',
'/content/drive/My Drive/Colab Notebooks/Sportsperson image classification supervised learningnng/Sports person
classifier/model/dataset/virat_kohli/6a1a57040d.jpg',
'/content/drive/My Drive/Colab Notebooks/Sportsperson image classification supervised learningnng/Sports person
classifier/model/dataset/virat_kohli/6ca67cbf33.jpg',
'/content/drive/My Drive/Colab Notebooks/Sportsperson image classification supervised learningnng/Sports person
classifier/model/dataset/virat_kohli/5e51e39db9.jpg',
'/content/drive/My Drive/Colab Notebooks/Sportsperson image classification supervised learningnng/Sports person
classifier/model/dataset/virat_kohli/2c06edccfb.jpg',
'/content/drive/My Drive/Colab Notebooks/Sportsperson image classification supervised learningnng/Sports person
classifier/model/dataset/virat_kohli/ee9b8b6b6b.jpg',
'/content/drive/My Drive/Colab Notebooks/Sportsperson image classification supervised learningnng/Sports person
classifier/model/dataset/virat_kohli/f4f9df9fcc.jpg',
'/content/drive/My Drive/Colab Notebooks/Sportsperson image classification supervised learningnng/Sports person
classifier/model/dataset/virat_kohli/e309859808.jpg',
'/content/drive/My Drive/Colab Notebooks/Sportsperson image classification supervised learningnng/Sports person
classifier/model/dataset/virat_kohli/ce8509c4a3.jpg',
'/content/drive/My Drive/Colab Notebooks/Sportsperson image classification supervised learningnng/Sports person
classifier/model/dataset/virat_kohli/688be3f972.jpg',
'/content/drive/My Drive/Colab Notebooks/Sportsperson image classification supervised learningnng/Sports person
classifier/model/dataset/virat_kohli/59a89ee258.jpg',
'/content/drive/My Drive/Colab Notebooks/Sportsperson image classification supervised learningnng/Sports person
classifier/model/dataset/virat_kohli/80b6d56dca.jpg',
'/content/drive/My Drive/Colab Notebooks/Sportsperson image classification supervised learningnng/Sports person
classifier/model/dataset/virat_kohli/da99f49cfd.jpg',
'/content/drive/My Drive/Colab Notebooks/Sportsperson image classification supervised learningnng/Sports person
classifier/model/dataset/virat_kohli/28e155755b.jpg',
'/content/drive/My Drive/Colab Notebooks/Sportsperson image classification supervised learningnng/Sports person
classifier/model/dataset/virat_kohli/f8d44f8840.jpg',
'/content/drive/My Drive/Colab Notebooks/Sportsperson image classification supervised learningnng/Sports person
classifier/model/dataset/virat_kohli/7953108375.jpg',
'/content/drive/My Drive/Colab Notebooks/Sportsperson image classification supervised learningnng/Sports person
classifier/model/dataset/virat_kohli/632b7bb5ea.jpg',
'/content/drive/My Drive/Colab Notebooks/Sportsperson image classification supervised learningnng/Sports person
```

```
classifier/model/dataset/virat_kohli/b1b30289cc.jpg',
'/content/drive/My Drive/Colab Notebooks/Sportsperson image classification supervised learning/Sports person
classifier/model/dataset/virat_kohli/ea43ba6899.jpg',
```

```
class_dict = {}
count = 0
for celebrity_name in celebrity_file_names_dict.keys():
    class_dict[celebrity_name] = count
    count = count + 1
class_dict
```

```
{'virat_kohli': 0,
'serena_williams': 1,
'roger_federer': 2,
'maria_sharapova': 3,
'lionel_messi': 4}
```

```
x = []
y = []

for celebrity_name, training_files in celebrity_file_names_dict.items():
    for training_image in training_files:
        img = cv2.imread(training_image)
        if img is None:
            continue
        scaled_raw_img = cv2.resize(img, (32,32))
        img_har = w2d(img, 'db1', 5)
        scaled_img_har = cv2.resize(img_har, (32,32))
        combined_image= np.vstack((scaled_raw_img.reshape(32*32*3,1),scaled_img_har.reshape(32*32,1)))
        x.append(combined_image)
        # Convert celebrity name to numerical label using class_dict
        y.append(class_dict[celebrity_name])
```

```
len(x)
```

```
303
```

```
len(x[0])
```

```
4096
```

```
x = np.array(x).reshape(len(x), 4096).astype(float)
x.shape
```

```
(303, 4096)
```

```
x[0]
```

```
array([248., 249., 244., ..., 141., 68., 66.]
```

```
y[0]
```

```
0
```

Train the Model

Used SVM with rbf kernel tuned with heuristic finetuning

```
from sklearn.svm import SVC
from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import train_test_split
from sklearn.pipeline import Pipeline
from sklearn.metrics import classification_report
```

```
x_train, x_test, y_train, y_test = train_test_split(x, y, random_state=0, stratify=y)
```

```
pipe = Pipeline([('scaler', StandardScaler()), ('svc', SVC(kernel='rbf', C=10))])
pipe.fit(x_train, y_train)
pipe.score(x_test, y_test)
```

```
0.5263157894736842
```

```
len(x_test)
```

```
76
```

```
print (classification_report(y_test, pipe.predict(x_test)))
```

	precision	recall	f1-score	support
0	0.51	0.92	0.66	25
1	0.75	0.27	0.40	11
2	0.75	0.25	0.38	12
3	0.47	0.47	0.47	15
4	0.50	0.31	0.38	13
accuracy			0.53	76
macro avg	0.60	0.44	0.46	76
weighted avg	0.57	0.53	0.49	76

Grid seacr cv

```
from sklearn import svm
from sklearn.ensemble import RandomForestClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.pipeline import make_pipeline
from sklearn.model_selection import GridSearchCV
```

```
model_params = {
    'svm': {
        'model': svm.SVC(gamma='auto', probability=True),
        'params': {
            'svc__C': [1,10,100,1000],
            'svc__kernel': ['rbf', 'linear']
        }
    },
    'random_forest': {
        'model': RandomForestClassifier(),
        'params': {
            'randomforestclassifier__n_estimators': [1,5,10]
        }
    },
    'logistic_regression': {
        'model': LogisticRegression(solver='liblinear',multi_class='auto'),
        'params': {
            'logisticregression__C': [1,5,10]
        }
    }
}
```

```
scores = []
best_estimators = {}
import pandas as pd
for algo, mp in model_params.items():
    pipe = make_pipeline(StandardScaler(), mp['model'])

    clf = GridSearchCV(pipe, mp['params'], cv=5, return_train_score=False)
    clf.fit(x_train,y_train)
    scores.append({
        'model': algo,
        'best_score': clf.best_score_,
        'best_params': clf.best_params_
    })
    best_estimators[algo] = clf.best_estimator_

# Create and display the DataFrame after the loop has finished
df = pd.DataFrame(scores,columns=['model', 'best_score', 'best_params'])
display(df)
```

	model	best_score	best_params
0	svm	0.547150	{'svc__C': 10, 'svc__kernel': 'rbf'}
1	random_forest	0.405121	{'randomforestclassifier__n_estimators': 10}
2	logistic regression	0.462995	{'logisticregression__C': 1}

Generate code with df

New interactive sheet

[illegible]

0.5263157894736842

0.4605263157894737

0.5263157894736842

```
array([[23,  0,  0,  0,  2],
       [ 5,  3,  0,  3,  0],
       [ 5,  0,  3,  3,  1],
```

```
[ 5, 1, 1, 7, 1],  
[ 7, 0, 0, 2, 4]]
```

```
import seaborn as sn  
plt.figure(figsize = (10,7))  
sn.heatmap(cm, annot=True)  
plt.xlabel('Predicted')  
plt.ylabel('Truth')
```

```
Text(95.7222222222221, 0.5, 'Truth')
```

